RETAIL AI ASSISTANT(SMART-SHOPPER AI)

Architecture Document

Hybrid Agentic Design — Personal Shopper & Customer Support

1. Overview

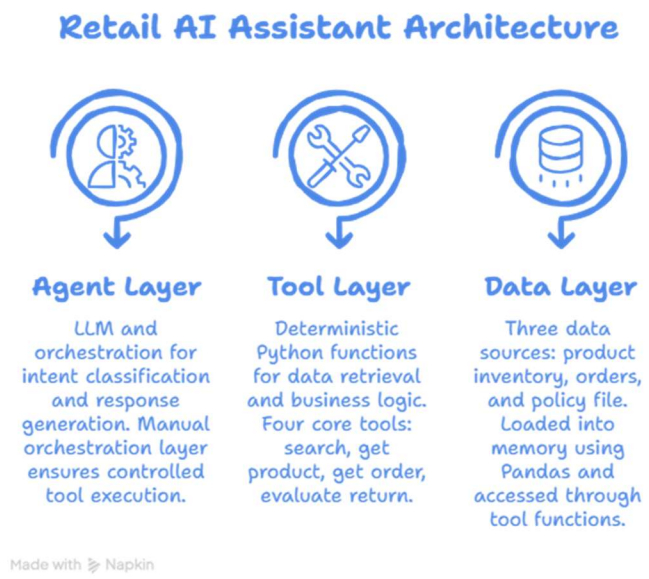
This project implements a simulation-based Retail AI Assistant capable of performing two distinct roles:

Role	Description
Personal Shopper	Acts as a Revenue Agent — recommends products based on user preferences and constraints
Customer Support	Acts as an Operations Agent — evaluates return eligibility using defined business policies

The system follows a hybrid agentic architecture, where:

- A language model is used for intent detection and explanation
- Deterministic Python functions (tools) handle data retrieval and business logic

This design ensures both flexibility (natural language understanding) and reliability (rule-based decisions).



2. System Architecture

The architecture is divided into three main layers, each with a clearly defined responsibility:

2.1 Agent Layer — LLM + Orchestration

The agent is implemented using a Groq-hosted LLM (llama-3.1-8-instant) and performs the following core functions:

- Intent classification — shopping vs. return requests
- Response generation — human-like, context-aware explanations
- Orchestration — coordinating tool calls based on detected intent

Since Groq does not natively support structured tool calling, a manual orchestration layer is implemented. This creates a controlled and interpretable agent pipeline:

STEP 1 LLM determines user intent from natural language	**STEP 2** Python logic selects and executes the appropriate tool
STEP 3 LLM explains tool output to the user in natural language	

2.2 Tool Layer — Deterministic Functions

The system defines four core tools, each operating on real CSV datasets with deterministic filtering and rule-based logic:

Tool Function	Description
search_products(filters)	Filters by size, price, sale status; ranks by bestseller_score
get_product(product_id)	Retrieves full product details from inventory
get_order(order_id)	Fetches purchase records and order metadata
evaluate_return(order_id)	Applies return eligibility rules deterministically

Product Recommendation Logic

The search_products tool applies these filters in sequence:

- Filters by size availability
- Applies price constraints
- Prioritizes sale items
- Uses bestseller_score for final ranking

Return Evaluation Logic

The evaluate_return tool enforces a strict hierarchy of business rules:

- Clearance items → not returnable (immediate rejection)
- Sale items → limited return window (store credit only)
- Vendor exceptions → exchange-only or extended return windows
- Standard items → 14-day return window

2.3 Data Layer

The system uses three structured data sources, all loaded into memory via Pandas and accessed exclusively through tool functions:

Data Source	Contents	Used By
Product Inventory CSV	Attributes, tags, stock, pricing	search_products, get_product
Orders CSV	Purchase records and order metadata	get_order, evaluate_return
Policy File	Return and exchange rules	evaluate_return

3. Design Decisions

3.1 Hybrid Agent Design

Due to the lack of native tool-calling support in Groq, a hybrid design ensures that the LLM handles understanding and explanation while Python tools handle execution and validation. This avoids over-reliance on the model while leveraging its full reasoning ability.

3.2 Separation of Concerns

The architecture cleanly separates three distinct responsibilities:

Layer	Responsibility	Benefit
Intent Detection	LLM interprets natural language input	Flexible, context-aware classification
Data Retrieval	Tools query structured CSV data	Reliable, fast, no hallucination
Business Rules	Tool logic enforces policy code	Predictable, auditable, compliant

3.3 Deterministic Business Logic

All critical operations — especially return decisions — are rule-based with no LLM involvement in eligibility decisions. This ensures:

- Predictability — same input always produces same output
- Policy compliance — rules are enforced strictly via code
- No ambiguity — decisions are transparent and explainable

4. Hallucination Prevention

The system minimizes hallucination through four complementary safeguards:

4.1	**Tool-Grounded Responses**	All factual outputs come from tools. Product recommendations are based on real inventory; return decisions from actual order and policy data. The LLM is only used to explain — never to generate facts.
4.2	**Restricted Data Access**	The LLM does not directly access CSV data and cannot fabricate product or order details. All information flows through controlled tool interfaces.
4.3	**Explicit Error Handling**	Invalid order ID → clear error message. Missing product → graceful fallback. No matching products → user is informed. This prevents incorrect assumptions.
4.4	**Deterministic Return Evaluation**	Return eligibility is calculated using fixed rules based on order date, product flags, and policy. No probabilistic or generative decision-making.

5. Tool Selection Strategy

5.1 Intent Detection

The system classifies user input into two categories using the LLM via a classification prompt:

- Shopping intent — triggers the Personal Shopper flow
- Return / Support intent — triggers the Customer Support flow

5.2 Execution Flows

Shopping Flow	Support Flow
① Detect shopping intent ② Extract filters (size, price, tags) ③ Call search_products(filters) ④ LLM explains recommendations	① Detect return intent ② Extract order ID from request ③ Call evaluate_return(order_id) ④ LLM explains eligibility decision

5.3 Controlled Tool Invocation

- Only predefined tools can be called — no arbitrary execution
- Tool outputs are validated before being passed to the LLM for explanation
- Strict separation between tool execution and model explanation

6. Limitations & Future Improvements

Area	Improvement Opportunity
Filter Extraction	Currently heuristic-based; can be improved with structured parsing or fine-tuned classifiers
Manual Orchestration	Can be replaced with native tool calling once supported by the provider
Ranking Logic	Can be enhanced with personalization signals, user history, and collaborative filtering